

Sesión 05
Líneas de Productos Software

Anass El Jabiry Kaddir

Departamento de Informática
Universidad de Almería

Almería, lunes 11 de noviembre de 2024

Índice de Contenidos

Página

1. Introducción		3
2. Desarrollo Paso a Paso		4
Paso 1: Creación del Proyecto GestionMascotas		4
Paso 2: Creación del Modelo de Datos		4
Paso 3: Creación del Gestor de Core Data		5
Paso 4: Creación del ViewModel.....		6
Paso 5: Implementación de ListaPersonasView.....		7
Paso 6: Añadiendo Mascotas a las Personas		8
Ejercicio 1: Implementación de AddMascotaView.....		9
3. Conclusiones y Reflexiones		10

1. Introducción

En esta quinta práctica se trabajó en la creación de una nueva aplicación denominada **Gestion-Mascotas**, que permite gestionar una lista dinámica de personas y sus mascotas. En esta ocasión, el objetivo principal fue implementar la **persistencia de datos** utilizando **Core Data**, el principal framework de Apple para la gestión y almacenamiento de datos de manera permanente. Además de repasar conceptos previos, se buscó crear una app desde cero con nuevas funcionalidades, añadiendo características de persistencia avanzada, manejo de relaciones entre entidades, y un diseño distinto al de las prácticas anteriores.

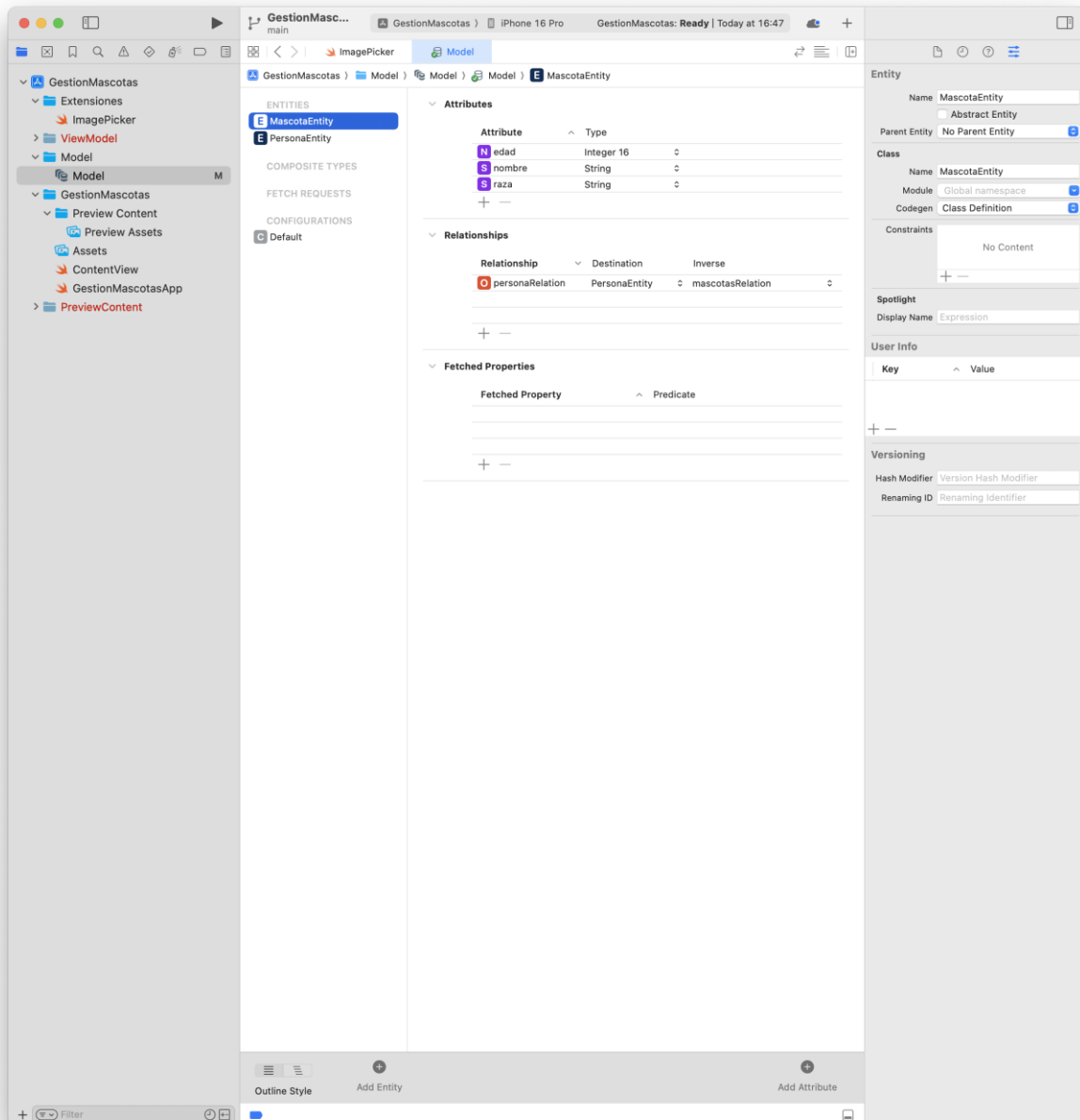
Herramientas Utilizadas

- **Xcode 13.4.1**: Entorno de desarrollo integrado utilizado para crear y depurar la aplicación.
- **Swift**: Lenguaje de programación con el que se desarrollaron todas las funcionalidades de la app.
- **SwiftUI**: Framework utilizado para la construcción de interfaces de usuario.
- **Core Data**: Framework empleado para implementar la persistencia de datos, permitiendo almacenar y recuperar la información de personas y mascotas.

2. Desarrollo Paso a Paso

Paso 1: Creación del Proyecto GestionMascotas

Se comenzó creando un nuevo proyecto en Xcode llamado **GestionMascotas**, el cual se configuró como una app de una sola vista (single-view). Aunque se pudo seleccionar la opción de uso de **Core Data** al crear el proyecto, en este caso se decidió no hacerlo para implementar manualmente el gestor y tener un mejor entendimiento del proceso.



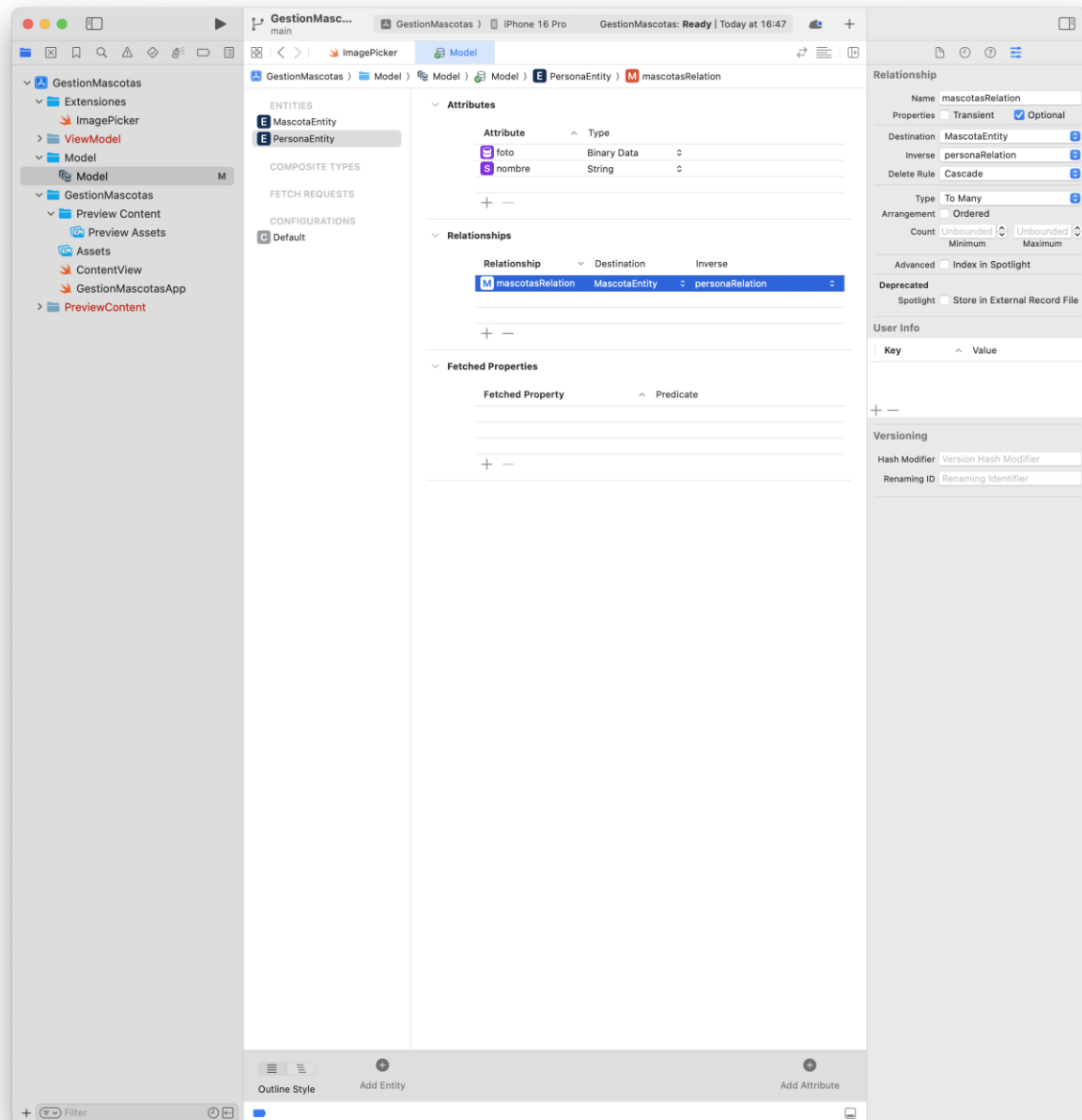
Captura 1: Proyecto inicial con la estructura básica de archivos en Xcode.

Paso 2: Creación del Modelo de Datos

Para gestionar los datos de personas y mascotas, se creó un modelo de datos llamado **GestionMascotasModel** utilizando **Core Data**. En este modelo se definieron dos entidades: **PersonaEntity** y **MascotaEntity**, con sus respectivos atributos y relaciones:

- **PersonaEntity**: Contiene los atributos nombre (de tipo String) y foto (de tipo Binary Data), representando la información básica de una persona.
- **MascotaEntity**: Contiene los atributos nombre (String), edad (Integer 16), y raza (String).

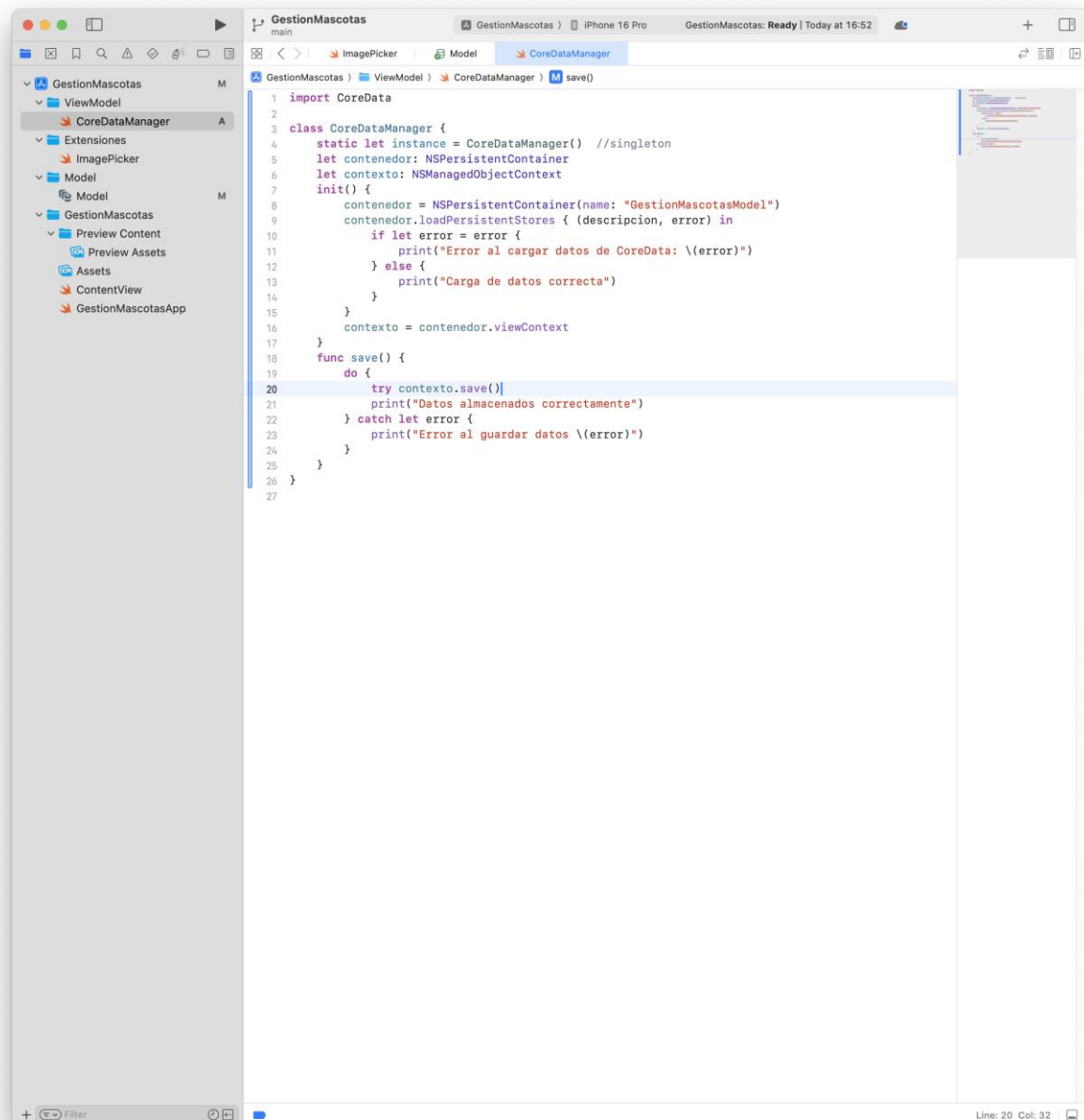
Se estableció una relación **1:N** entre **PersonaEntity** y **MascotaEntity**, indicando que una persona puede tener muchas mascotas, mientras que una mascota pertenece a una sola persona. Además, se configuró una **regla de eliminación en cascada** (Delete Rule: Cascade) para que, al eliminar una persona, todas sus mascotas sean eliminadas automáticamente.



Captura 2: Modelo de datos de Core Data con las entidades Persona y Mascota y sus relaciones.

Paso 3: Creación del Gestor de Core Data

Dentro del grupo **ViewModel**, se creó un archivo Swift llamado **CoreDataManager** que implementa un gestor para interactuar con Core Data. El gestor sigue el patrón de diseño **Singleton**, lo que garantiza que solo exista una instancia en la aplicación. Se incluyó un contenedor persistente (NSPersistentContainer) y un contexto (NSManagedObjectContext) para gestionar las operaciones de guardado y recuperación de datos.

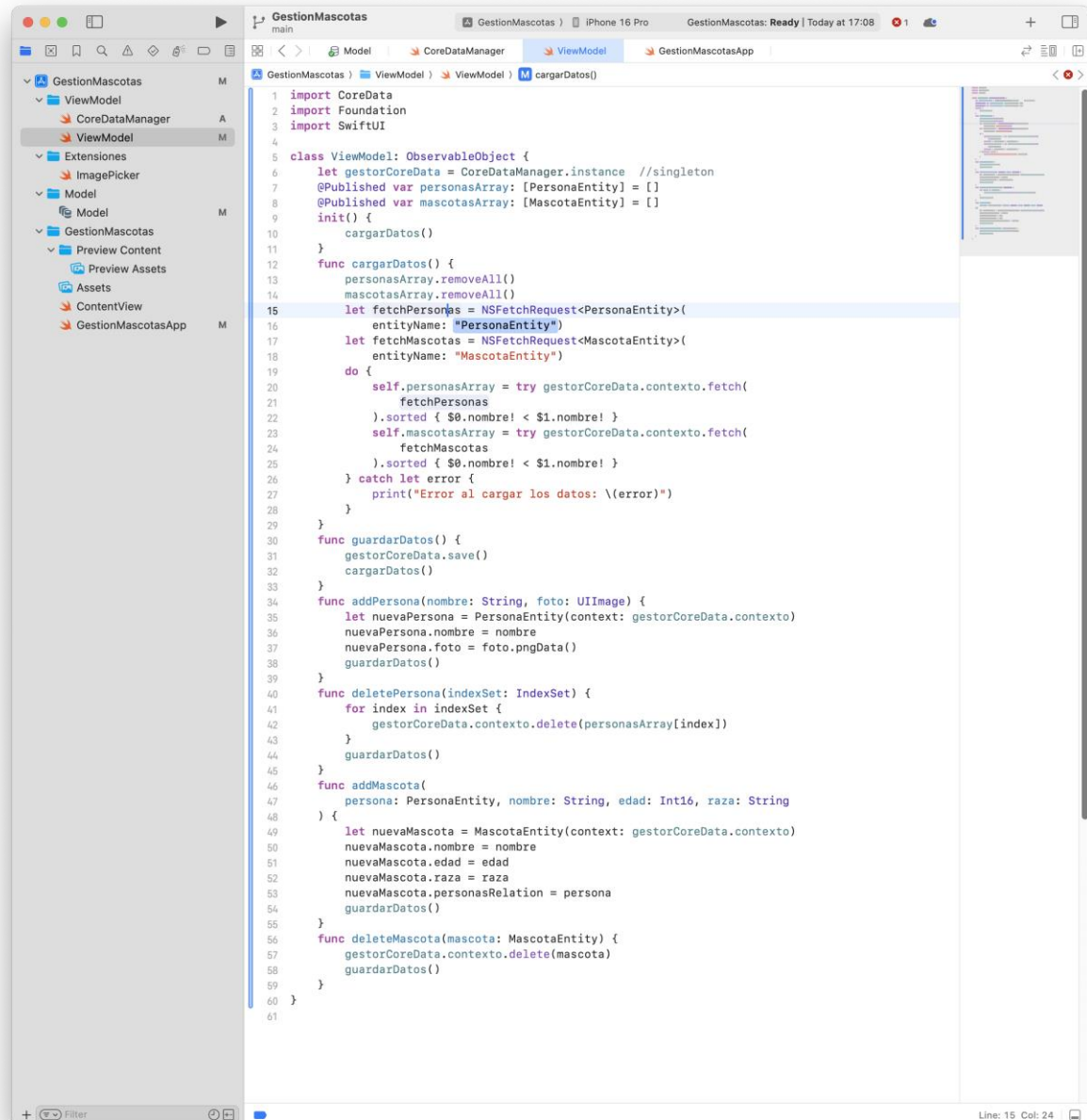


Captura 3: Código de la clase CoreDataManager, mostrando la inicialización del contenedor y la función save() para guardar datos en Core Data.

Paso 4: Creación del ViewModel

Se desarrolló una clase **ViewModel** que adopta el protocolo ObservableObject. Esta clase contiene un array de instancias de PersonaEntity y MascotaEntity y proporciona métodos para cargar, insertar y eliminar datos. Estos métodos permiten manipular los datos de manera controlada y evitar la duplicación de código.

- **Método cargarDatos():** Realiza una consulta (fetch) sobre las entidades y ordena los resultados alfabéticamente por nombre antes de cargarlos en los arrays correspondientes.
- **Métodos de modificación:** Se implementaron métodos para añadir (addPersona, addMascota) y eliminar (deletePersona, deleteMascota) personas y mascotas.

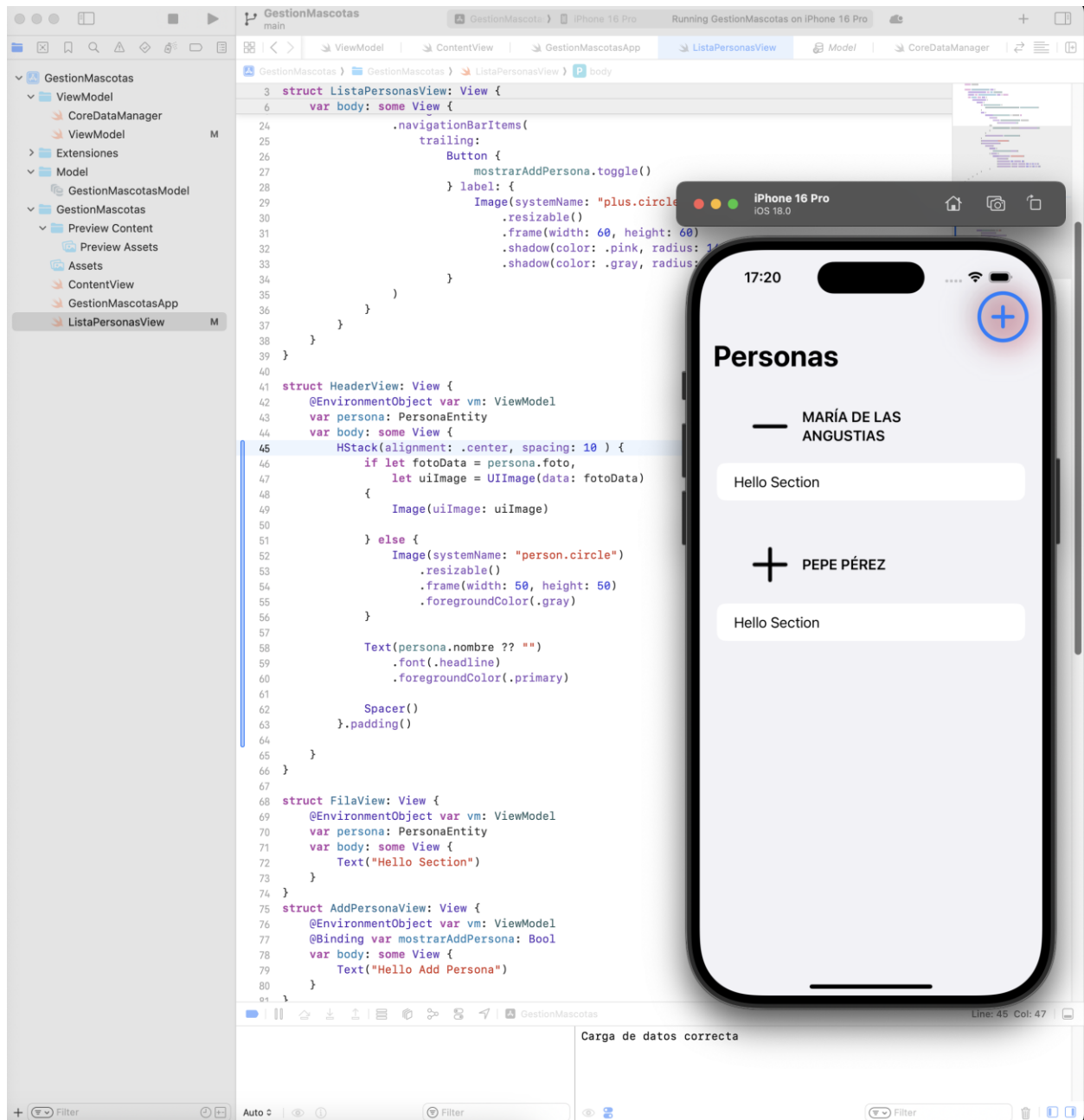


Captura 4: Implementación del ViewModel y sus métodos principales para la gestión de datos.

Paso 5: Implementación de ListaPersonasView

La vista principal de la aplicación es **ListaPersonasView**, que muestra una lista de personas almacenadas en Core Data. Cada sección de la lista representa a una persona, mostrando su imagen y nombre, junto con una lista de sus mascotas.

- **Navegación y Subvistas:** Se segmentó la funcionalidad en distintas subvistas (HeaderView, FilaView, AddPersonaView) para mantener el código organizado y facilitar el desarrollo incremental.
- **Botón para Añadir Personas:** Se implementó un botón que permite añadir nuevas personas a la lista mediante una subvista (AddPersonaView) que contiene un formulario para ingresar el nombre y una foto de la persona.

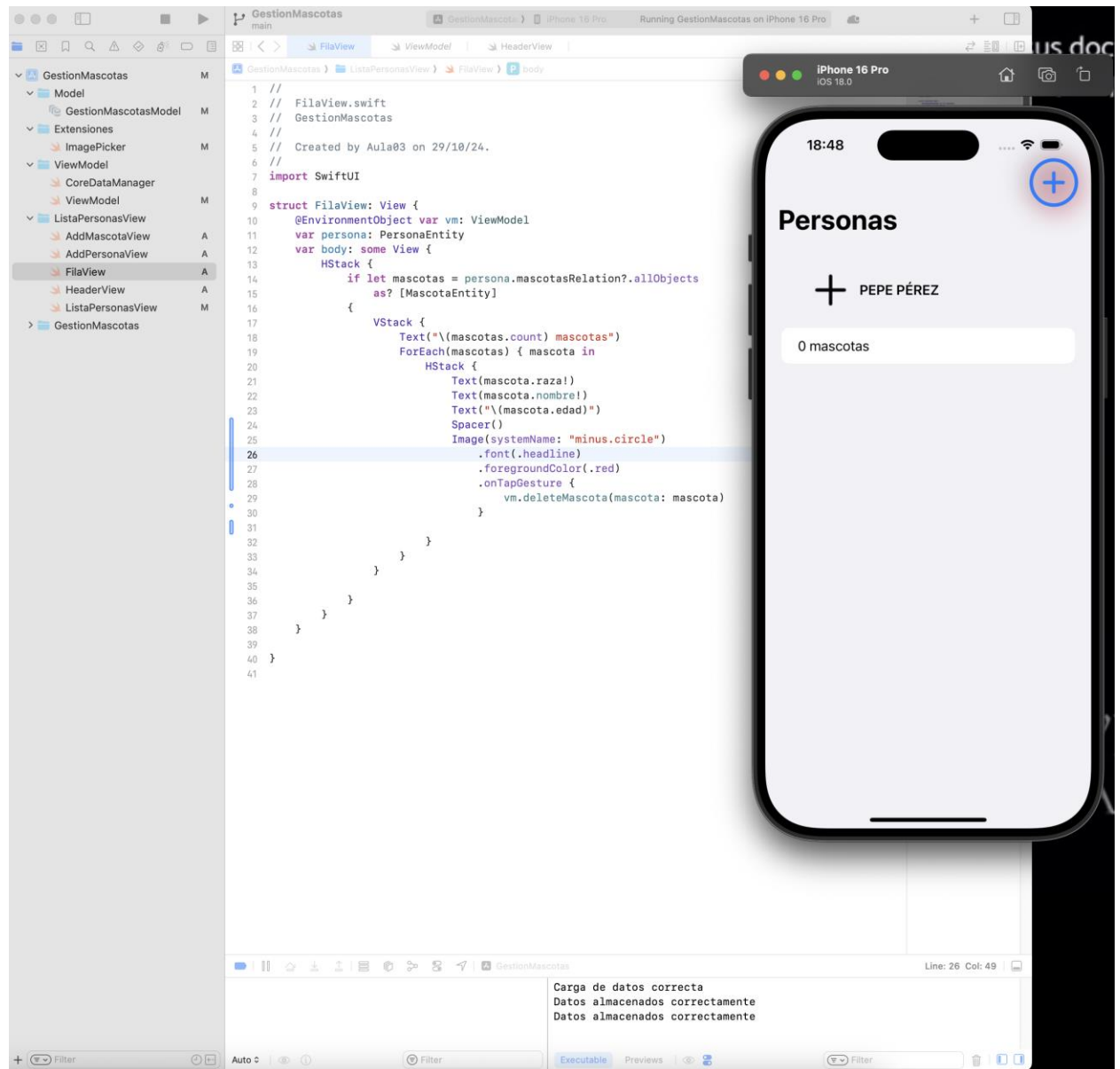


Captura 5: ListaPersonasView mostrando varias personas, cada una con su sección y una lista de sus mascotas.

Paso 6: Añadiendo Mascotas a las Personas

Para añadir mascotas, se utilizó el botón asociado a la imagen de la persona. Al hacer tap sobre la imagen, se crea una nueva mascota asociada a esa persona, con datos predefinidos (nombre generado aleatoriamente, edad fija, y raza fija). Esta funcionalidad se implementó para facilitar la prueba rápida de la app, aunque se planificó añadir un formulario completo para la inserción de mascotas más adelante.

- **Ejercicio 1:** Como ejercicio, se solicitó implementar la vista **AddMascotaView**, que permita al usuario especificar el nombre, la edad y la raza de la mascota, además de agregar la opción de una foto. Se proporcionaron pistas para el uso de componentes como Slider y Picker para permitir al usuario elegir la edad y la raza.



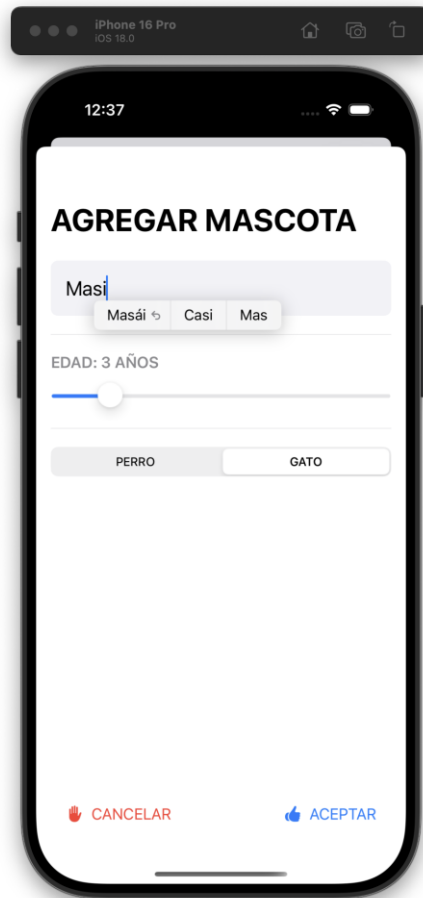
Captura 6: Añadiendo una mascota a una persona mediante el evento `.onTapGesture` en la imagen de la persona.

Ejercicio 1: Implementación de AddMascotaView

El ejercicio consistió en completar la vista **AddMascotaView** para que el usuario pueda especificar detalles de una nueva mascota. La vista se estructuró como un formulario con los siguientes componentes:

- **TextField:** Para ingresar el nombre de la mascota.
- **Slider:** Para seleccionar la edad de la mascota. Se utilizó un Slider con valores de tipo Double que posteriormente se convierten a Int16 para almacenar la edad.
- **Picker:** Un Picker con estilo SegmentedPickerStyle que permite al usuario elegir la raza de la mascota.

Finalmente, se incluyó un botón para guardar los datos ingresados y añadir la mascota al registro de la persona seleccionada.



Captura 7: AddMascotaView mostrando el formulario completo para agregar una nueva mascota.

3. Conclusiones y Reflexiones

En esta práctica aprendimos a utilizar **Core Data** para la persistencia de datos, lo cual nos permitió almacenar de manera permanente la información de personas y mascotas en nuestra aplicación. La implementación de un **gestor de Core Data** siguiendo el patrón **Singleton** facilitó la interacción entre el modelo de datos y la interfaz de usuario.

El diseño del **ViewModel** ayudó a mantener el código limpio y evitar la repetición innecesaria, mientras que la segmentación de las vistas permitió una mejor organización del proyecto. A través de esta práctica, se comprendió la importancia de la persistencia y la gestión adecuada de los datos, así como la necesidad de pensar en un diseño flexible y escalable para futuras mejoras.

Esta práctica nos acercó a la construcción de una app más completa, con una interfaz funcional y almacenamiento de datos robusto, estableciendo una base sólida para seguir creciendo en complejidad y funcionalidad.